

仅偏向于满足用户目标而不充分考虑系统目标，那么操作系统很可能实现不出来，即使实现出来也很难正常运行。事实上，在操作系统发展的 60 多年历史中，曾多次出现因过于强调各种极致能力而导致设计结构不合理并最终失败的案例。例如，1991 年到 1995 年间，IBM 雄心勃勃地投入了 20 亿美元，试图打造 Workplace 操作系统。然而，由于目标过于宏伟，系统过于复杂，最终导致项目失败^[1]。历史上的例子还有苹果的 Copland 操作系统项目^①、微软的 Windows VISTA 操作系统^②等。

本章将主要围绕操作系统的设计方法和典型的操作系统结构展开介绍，从而帮助读者理解现代操作系统中的一些基础设计方法与架构。

2.1 操作系统的机制与策略

本节主要知识点

- 如何控制操作系统的复杂性？
- 什么是机制？什么是策略？为什么要将二者尽可能分离？
- 操作系统内核有哪些典型的架构？

如何有效地控制操作系统设计的复杂性呢？操作系统乃至计算机系统中控制复杂性的一个重要设计原则是将策略与机制相分离^[2]。其中，策略（Policy）表示“要做什么”，机制（Mechanism）表示“如何做到”。如表 2.2 所示，对于计算机登录认证系统而言，机制包括输入处理、策略文件管理、桌面启动加载等，策略则包括什么用户、以什么权限登录等。另一个例子是操作系统的调度系统，策略包括先到先服务（First-Come-First-Service）、时间片轮转（Round Robin）、最短截止时间优先（Earliest Deadline First）、完全公平调度（Completely Fair Scheduling）等，机制则包括调度队列的设计、调度实体的表示（如线程）与时钟的中断处理等。

表 2.2 操作系统中典型的策略与机制

	策略	机制
登录	什么用户、以什么权限登录等	输入处理、策略文件管理、桌面启动加载等
调度	先到先服务、时间片轮转等	调度队列的设计、调度实体的表示、时钟的中断处理等

通过分离机制与策略，操作系统一方面可以通过设置多种不同的策略来适应不同的应用需求，而不需要重新实现对应的具体机制；另一方面也可以通过持续优化具体的机制来不断完善对具体策略的支持。

① 参见 [https://en.wikipedia.org/wiki/Copland_\(operating_system\)](https://en.wikipedia.org/wiki/Copland_(operating_system))。
② 参见 <https://medium.com/@benbob/what-really-happened-with-vista-an-insiders-retrospective-f713ee77c239>。

2.2 操作系统复杂性的管理方法

本节主要知识点

- 什么是 M.A.L.H 方法？
- 分层与层级有哪些区别？

管理系统复杂性的重要方法是 M.A.L.H 方法^[3]，即模块化（Modularity）、抽象（Abstraction）、分层（Layering）和层级（Hierarchy）。

模块化。模块化是通过分而治之（Divide and Conquer）原则，将一个复杂系统分解为一系列通过明确定义的接口进行交互的模块，并严格保障模块之间的界限。模块划分并不是越细越好，过多的模块反而会因为模块之间联系过多而无益于复杂性的控制；划分要充分考虑高内聚和低耦合，使模块具有较好的独立性。现代操作系统都存在一定程度的模块化结构，包括进程管理、内存管理、网络协议栈、设备驱动等。

抽象。抽象是在模块化的基础上将接口与内部实现分离，从而使模块之间只需通过抽象的接口进行相互调用，而无须关心各个模块间的内部实现。设计良好的抽象应该尽可能依从模块间的自然边界，并尽可能减少模块间的交互，从而减少错误在模块间的传递，提高整体系统的开发效率、质量与性能等。例如，UNIX 系列操作系统所提供的虚拟内存为物理内存提供了良好的抽象，使应用程序无须关心物理地址的具体位置，而只需要针对独立的、连续的虚拟地址空间进行设计。同样，文件系统抽象使得应用程序无须关心数据在物理介质（如闪存或磁盘）中的具体位置，而只需要通过定义好的文件系统接口（如 open、read、write 等）操作相应的数据。

小知识：宽进严出原则

模块的接口应该容忍各种可能的输入，抑制错误甚至恶意的输入，避免错误或恶意输入的效果在模块内传播，并且尽可能严格地控制模块对外的输出，从而减少错误在模块间的传播。

良好的模块化与抽象可以很好地将一个大系统分解为一系列交互较好的模块。然而，对于一个大型复杂系统而言，一个可能的结果是分解的模块非常多，相互的交互关系非常复杂（如图 2.1 所示），因此需要进一步通过分层与层级来控制复杂性。

分层。分层是通过将模块按照一定的原则进行层次划分，约束每层内模块间与跨层次模块间的交互方式，从而有效地减少模块之间的交互。通常的原则是：一个模块只能和同层模块以及相邻的上层或下层模块进行交互，而不能跨层和再上一层或再下一层的模块进行交互。分层也是构建复杂系统的一个重要方式：确定层级后，我们可以先构建

底层的模块，然后利用底层模块提供的功能与服务进一步构建上层的模块。例如，第 1 章介绍操作系统的简明结构时，也是通过将操作系统分为内核、系统服务、应用框架与应用程序等层次来控制操作系统的复杂性。

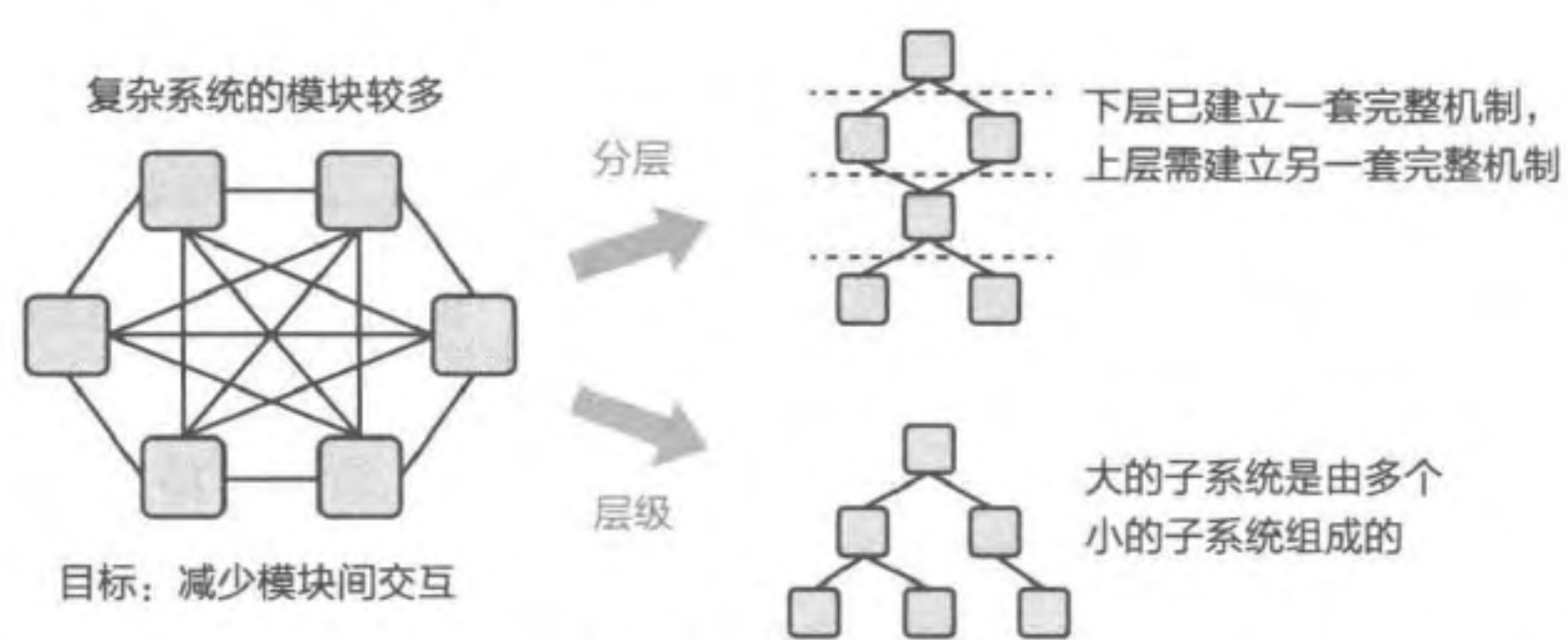


图 2.1 过多的模块会导致交互非常复杂，需要通过分层与层级来降低复杂性

小知识：分层

“计算机科学中的所有问题都可以通过另外一个分层来解决，除了太多间接分层的问题外。”（All problems in computer science can be solved by another level of indirection, except for the problem of too many layers of indirection.）

——Butler Lampson 1992 年图灵奖获奖报告（引自 David Wheeler）

对于一个已经构建好的复杂系统，分层也是一个很好的解决问题的方式。例如，针对数据中心的服务器性能越来越强大但利用率低下且管理困难等问题，系统虚拟化通过在操作系统之下再构建一层虚拟机监控器层，有效地提升了资源利用率，提高了性能与错误隔离的能力，并且降低了管理难度。因而，系统虚拟化也成为云计算的关键支撑技术之一。

层级。层级是另外一种模块的组织方式：首先将一些功能相近的模块组成一个具有清晰接口的自包含子系统，然后再将这些子系统递归式地组成一个具有清晰接口的更大的子系统。层级是日常生活中常见的组织形式。例如，在公司的组织架构中，一个经理管理一组成员，一组经理构成一个部门，多个部门构成一个事业部，多个事业部构成一个公司。在操作系统中，虚拟内存是一个子模块，与物理内存分配、缺页异常处理、页换入换出等一同构成内存管理模块，内存管理模块再与进程管理模块、设备驱动模块等一起构成操作系统内核。

分层与层级

分层和层级这两个概念有点像，有些时候不容易区分。简要而言，分层是指不同类模块之间的层次化，而层级则是指同类模块之间的分层。两种方法可以组合起来一

起降低模块的复杂性。例如，操作系统包括内核层、系统服务层与应用框架层等，而操作系统内核中则可以通过如上的层级方式进行组织。

M.A.L.H 是操作系统中降低复杂性与组织各种功能模块的有效方法。早期的操作系统（如 MS-DOS）缺乏模块化能力，不区分用户程序和操作系统的运行环境，使得一个应用的错误就可能导致整个系统崩溃。后来，操作系统设计人员利用处理器提供的特权级^①对系统进行分层以避免错误的传播，并进一步针对各种不同的需求设计了各种不同的操作系统架构。

2.3 操作系统内核架构

本节主要知识点

- 操作系统的内核有哪些常见的架构？
- MS-DOS 的架构有什么优点和缺点？
- 一个应用能否运行在多个内核之上？
- 微内核与宏内核到底孰优孰劣？

随着操作系统功能的不断增多和代码规模的不断扩大，提供合理的层级结构，对于降低操作系统的复杂性、提升操作系统的安全性与可靠性来说变得尤为重要。图 2.2 列举了一些常见的操作系统内核架构，下面我们对这些架构进行简要的介绍和分析。

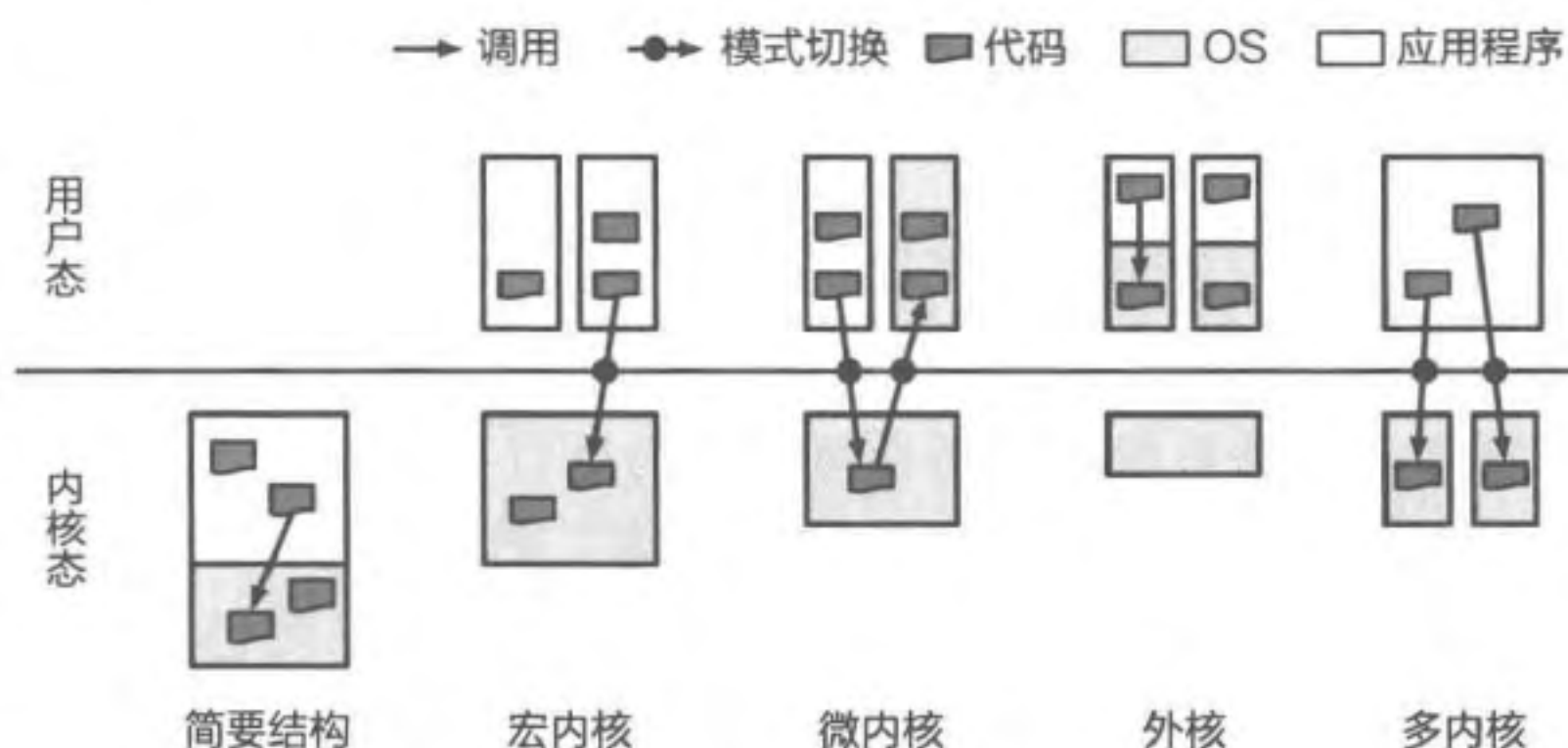


图 2.2 操作系统内核架构的频谱：简要结构（如 MS-DOS）、宏内核（如 UNIX/Linux）、微内核、外核与多内核等

① 处理器特权级指用户态与内核态，后文会深入介绍。